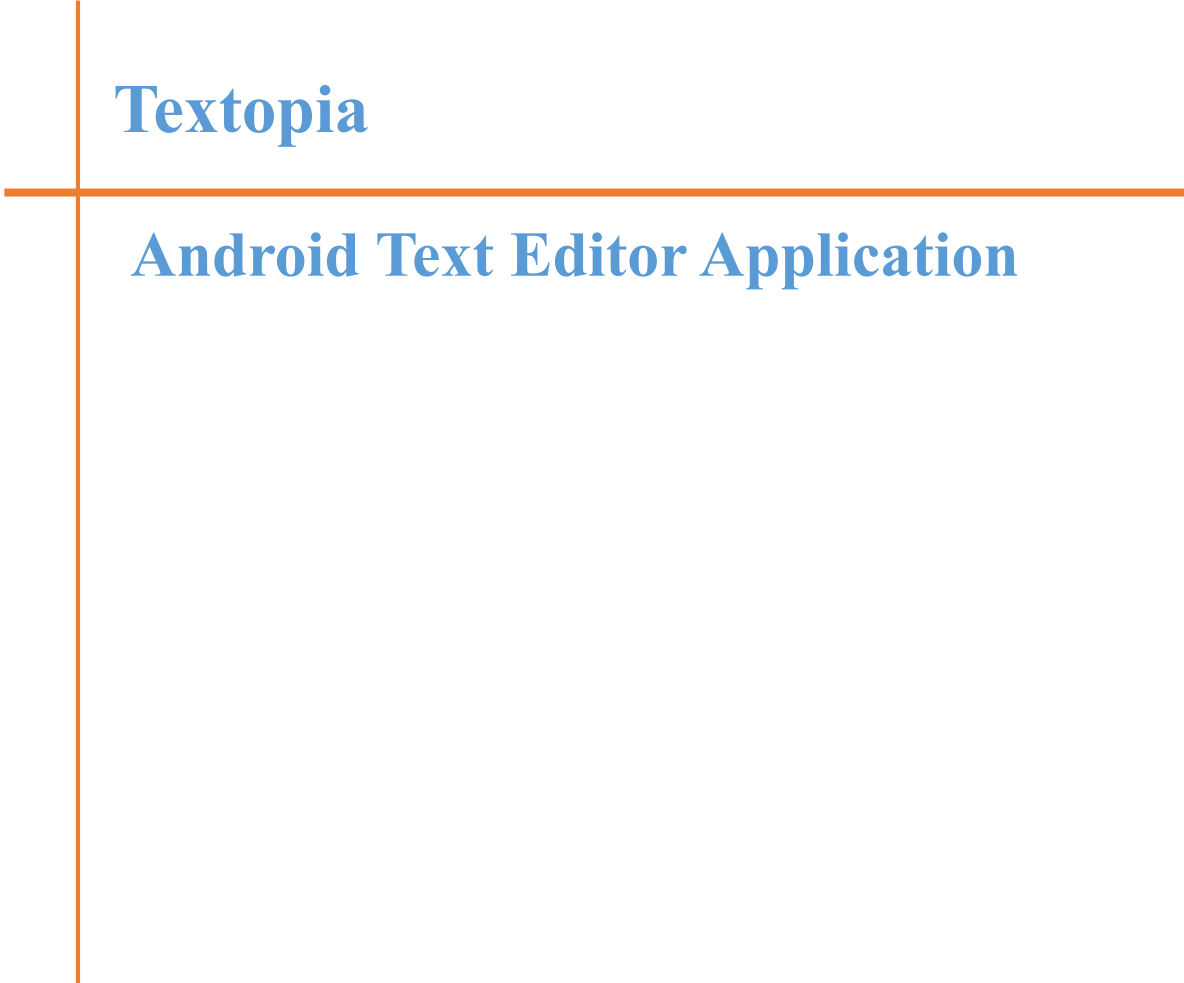


A decorative graphic consisting of a vertical orange line and a horizontal orange line intersecting to form a crosshair. The vertical line is positioned to the left of the word 'Textopia', and the horizontal line is positioned below it, extending across the width of the page.

Textopia

Android Text Editor Application

**IS2205 Mobile Application Design and Development
Assignment**

Project Documentation

Table of Contents

1. Project Overview	3
1.1 Key Objectives	3
1.2 Demo Video	3
2. Features Implementation.....	4
2.1 Basic Editor Functionality.....	4
2.1.1 File Operations	4
2.1.2 Text Editing Operations.....	5
2.1.3 Supported File Types:.....	7
2.2 Default Kotlin Highlighting	8
2.2.1 Kotlin Syntax Support	8
2.2.2 Configurable Syntax Highlighting.....	8
2.2.3Implementation.....	10
2.2.4 Configuration System	10
2.3 ADB Connection & Kotlin Compiler Integration	11
2.3.1 Compilation Process:.....	11
2.3.2 Kotlin-Specific Features	11
2.3.4 Error Handling and Reporting	11
2.4 Integration and Error Handling	12
2.4.1 Robust Error Management.....	12
2.4.2 Compilation Status Display	12
3. Technical Architecture.....	12
3.1 Development Environment	12
3.2 Key Libraries and Dependencies.....	12
4. Usage Guide.....	13
4.1 Creating and Opening Files.....	13
4.2 Text Editing Operations	13
4.3 Syntax Highlighting	13
4.4 Code Compilation	13
4.5 Advanced Features	13
5. Team Members and Contributions.....	14
6. Conclusion	15

1. Project Overview

- **Project Name:** Textopia
- **Platform:** Android
- **Target Android Version:** Android 9 and above
- **Primary Purpose:** A comprehensive multi-language text editor with advanced syntax highlighting capabilities and specialized Kotlin compilation features.

Textopia is a feature-rich Android text editor application that supports multiple programming languages and file formats. While it includes specialized features for Kotlin development including compilation capabilities, the application serves as a versatile text editor for various programming languages including Java, Python, JavaScript, C++, and plain text files.

1.1 Key Objectives

- Provide a robust multi-language text editing environment on Android devices
- Support comprehensive file extension handling for various programming languages
- Implement native Kotlin syntax highlighting with compilation capabilities
- Enable configurable syntax highlighting for multiple programming languages
- Integrate ADB-based compilation specifically for Kotlin development
- Deliver an intuitive and responsive user interface for all supported file types

1.2 Demo Video

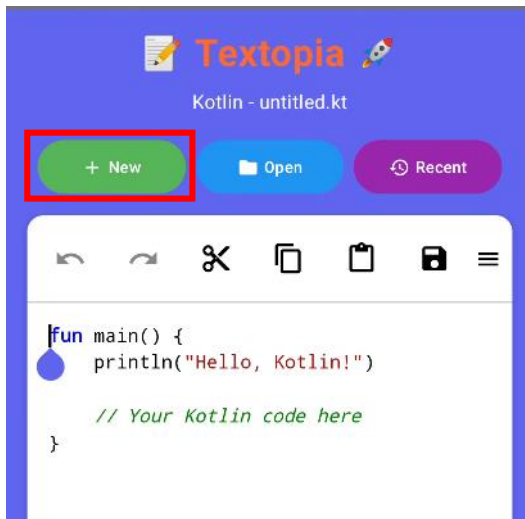
<https://drive.google.com/file/d/1sST3VihgshLYE-vqPi0PZluuWqCWOARK/view?usp=drivesdk>

2. Features Implementation

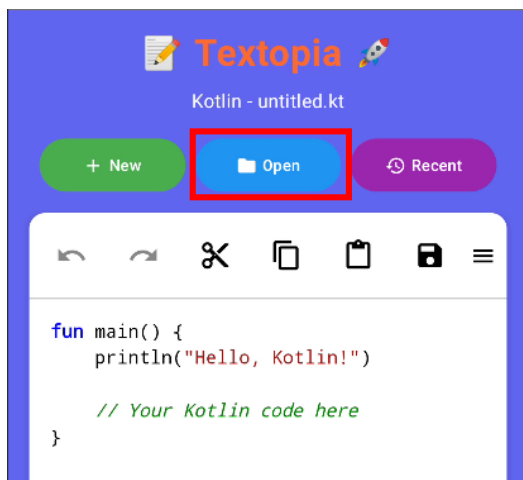
2.1 Basic Editor Functionality

2.1.1 File Operations

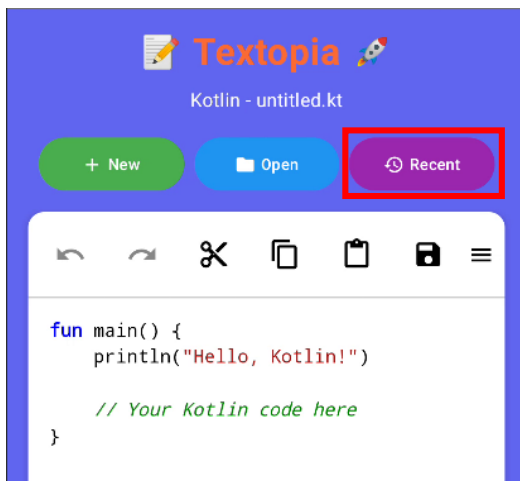
- **Create New Files:** Users can start new files and choose from multiple file types.



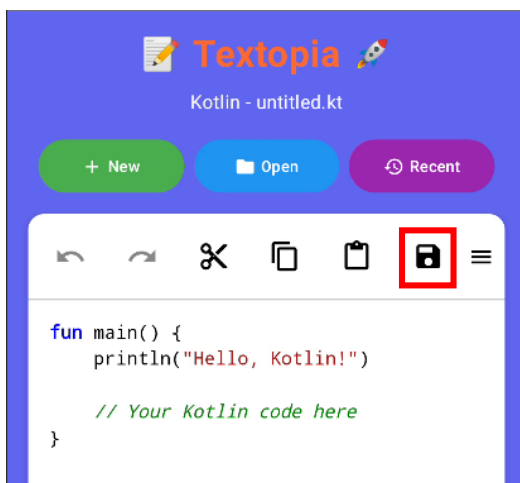
- **Open Files:** Open existing files from device storage. The app automatically detects the file type.



- **Recent Files:** Opens the recently saved files using this applications.

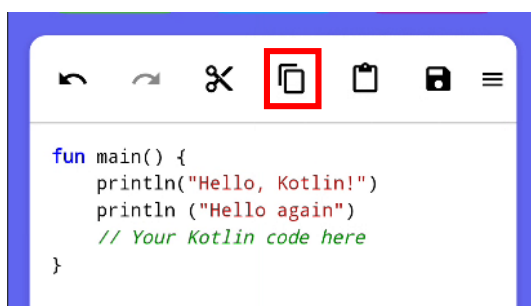


- **Save Files:** Supports manual and automatic saving, keeping the correct file extension.

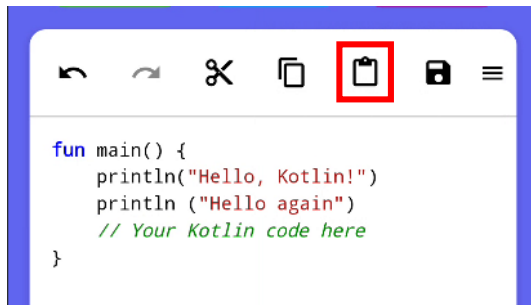


2.1.2 Text Editing Operations

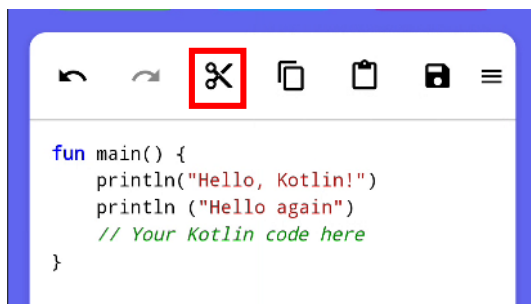
- Copy with Android clipboard support.



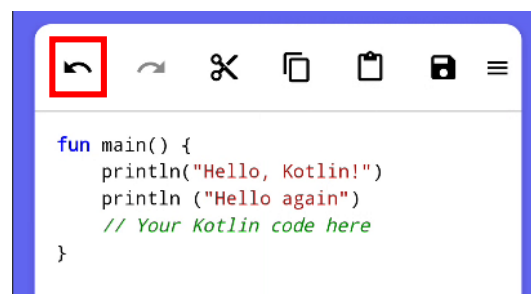
- Paste with Android clipboard support.



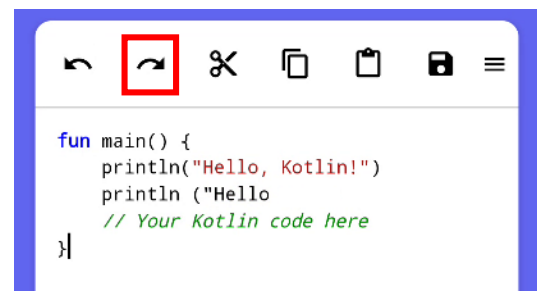
- Cut with Android clipboard support.



- Undo and redo with multiple steps.

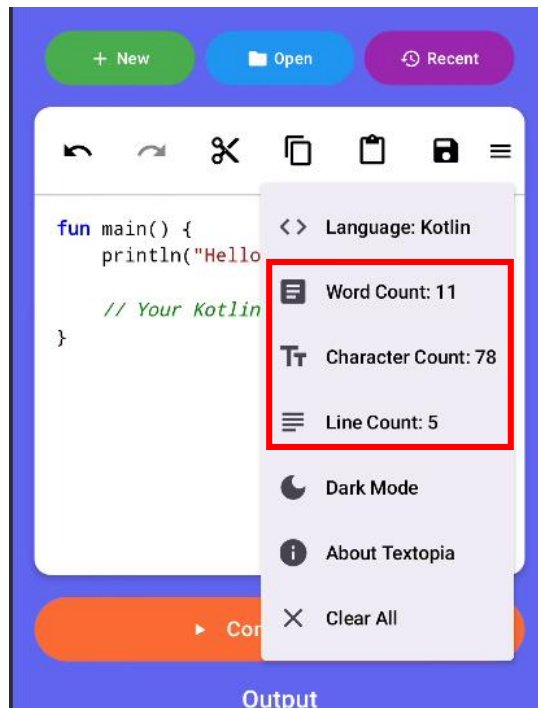


Undo



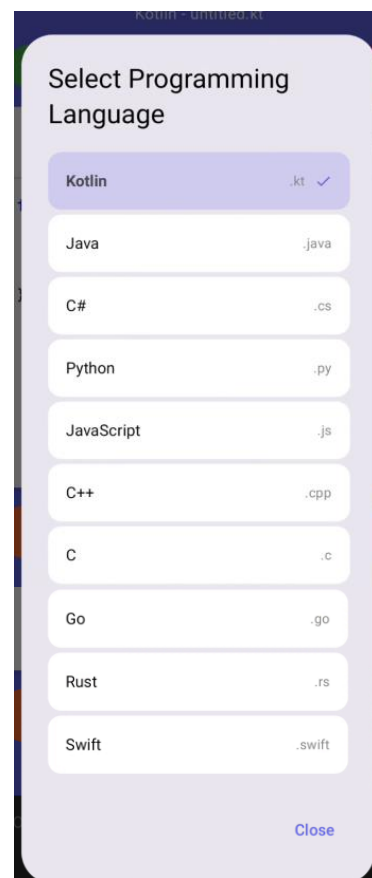
Redo

- Shows live character, word and line counts.



2.1.3 Supported File Types:

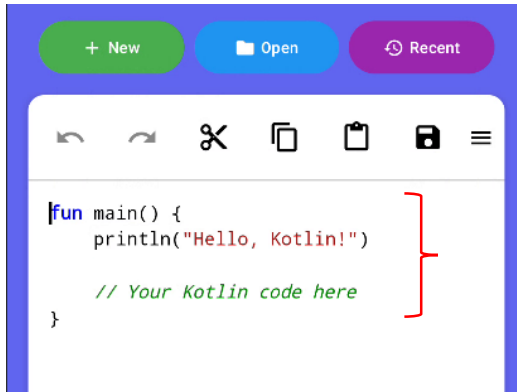
- ✓ **Kotlin:** .kt
- ✓ **Java:** .java
- ✓ **C#:** .cs
- ✓ **Python:** .py
- ✓ **JavaScript:** .js
- ✓ **C++:** .cpp
- ✓ **C:** .c
- ✓ **Go:** .go
- ✓ **Rust:** .rs
- ✓ **Swift:** .swift



2.2 Default Kotlin Highlighting

2.2.1 Kotlin Syntax Support

- Built-in syntax highlighting for Kotlin, optimized for editing and compiling.
- Highlights keywords, comments, strings, functions, data types, operators, annotations, and control flow.

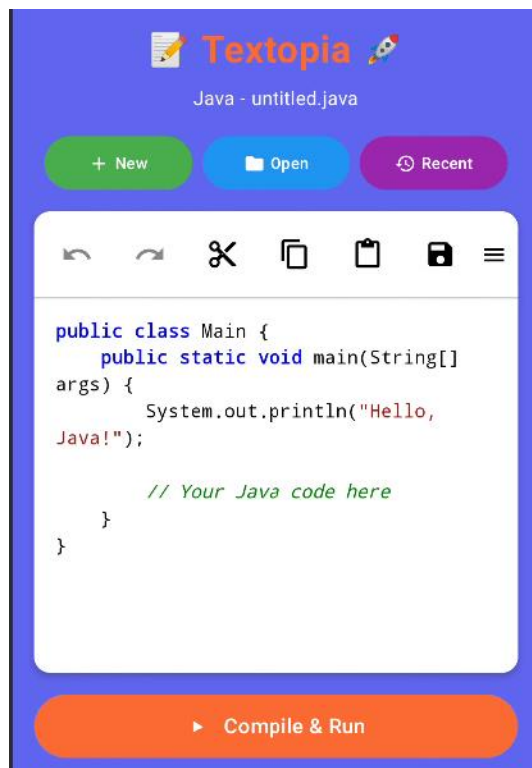


2.2.2 Configurable Syntax Highlighting

This is a Multi-Language Support System. Textopia's configurable highlighting system supports a wide range of programming languages through external configuration files, making it a truly versatile text editor.

Supported Languages through Configuration

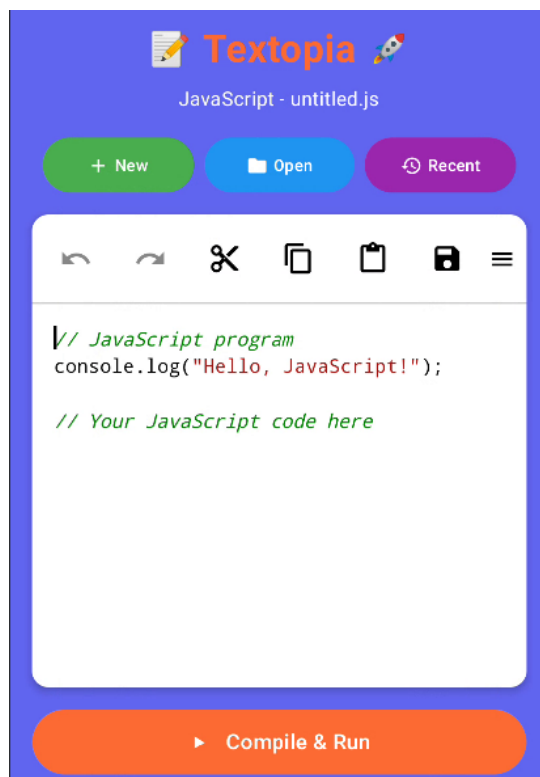
- Web Development: JavaScript
- System Programming: C, C++, Rust, Go
- Scripting: Python
- Enterprise: Java, C#



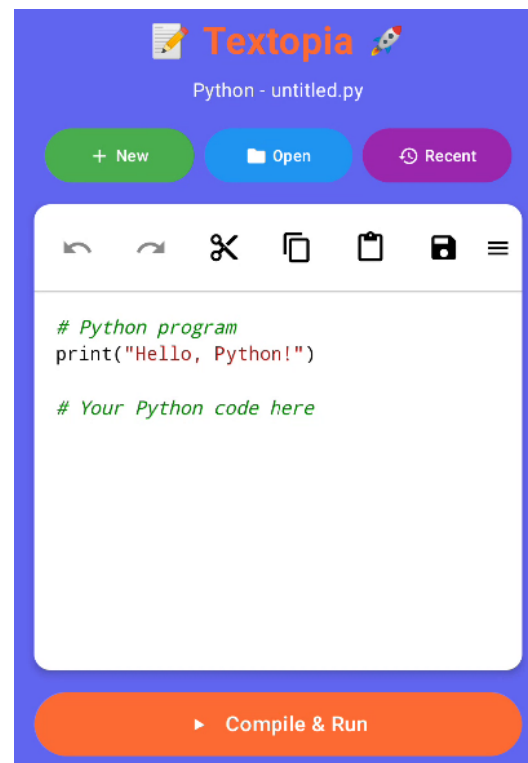
Java



C#



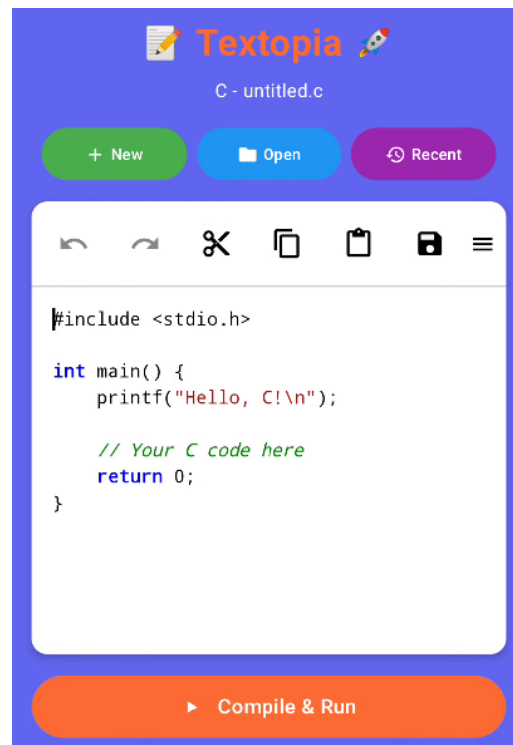
JavaScript



Python



C++



C

2.2.3 Implementation

- Highlights code in real-time, including nested structures and string interpolation.
- Built using Android's EditText with enhancements for multiple languages.
- Automatically detects file types and applies the correct editor settings.
- Real-time character and word count using a custom TextWatcher.
- File operations use Android's Storage Access Framework for secure handling.

2.2.4 Configuration System

- JSON/XML-based Configuration: Syntax rules defined in structured format for easy modification
- File Extension Mapping: Automatic language detection based on file extensions
- Rule Definition: Keywords, comments, strings, operators, and special constructs configurable per language

- **Dynamic Loading:** Configuration files are loaded at runtime and applied immediately
- **Fallback System:** Default highlighting for unrecognized file types

2.3 ADB Connection & Kotlin Compiler Integration

While Textopia supports editing multiple file types, the compilation feature is specifically designed for Kotlin files, providing a complete Kotlin development environment on Android.

2.3.1 Compilation Process:

- **ADB Connection:** Establishes secure connection to desktop machine running Kotlin compiler
- **Compile Button:** Prominent "Compile & Run" button appears when editing Kotlin files
- **Command Execution:** Sends Kotlin-specific compilation commands through ADB bridge
- **Output Display:** Results shown in dedicated output section with syntax-aware formatting

2.3.2 Kotlin-Specific Features

- **Kotlin Script Support:** Enhanced support for .kts (Kotlin Script) files
- **Dependency Management:** Basic support for common Kotlin libraries
- **Error Parsing:** Kotlin compiler error message parsing and highlighting

2.3.4 Error Handling and Reporting

- **Real-time Error Display:** Compilation errors highlighted in the editor
- **Status Reporting:** Clear indication of compilation success or failure
- **Error Parsing:** Compiler output parsed to extract meaningful error information
- **User Feedback:** Visual indicators and notifications for compilation status

2.4 Integration and Error Handling

2.4.1 Robust Error Management

- Exception Handling: Comprehensive try-catch blocks for all critical operations
- User-Friendly Messages: Clear error messages for different failure scenarios
- Graceful Degradation: App continues functioning even when some features fail
- Connection Stability: Automatic retry mechanisms for ADB connections

2.4.2 Compilation Status Display

- Visual Indicators: Loading animations during compilation
- Success/Failure States: Clear visual distinction between different states
- Progress Tracking: Step-by-step compilation progress when possible

3. Technical Architecture

3.1 Development Environment

- Programming Language: Kotlin
- IDE: Android Studio
- Minimum SDK: API 28 (Android 9)
- Target SDK: API 34 (Android 14)
- Build System: Gradle

3.2 Key Libraries and Dependencies

- Android Jetpack Components: ViewModel, LiveData, Navigation
- File I/O: Android Storage Access Framework
- Network Communication: Custom ADB protocol implementation
- UI Components: Material Design Components
- Text Processing: Custom syntax highlighting engine

4. Usage Guide

4.1 Creating and Opening Files

- New File: Tap the "New" button and specify file name and extension
- Open File: Use "Open" button to browse and select existing files
- Recent Files: Access recently opened files through "Recent" button

4.2 Text Editing Operations

- Basic Editing: Tap in the editor area and start typing
- Selection: Long press and drag to select text
- Clipboard Operations: Use toolbar buttons for copy, cut, and paste
- Undo/Redo: Use arrow buttons in toolbar for operation history

4.3 Syntax Highlighting

1. Automatic Detection: File extension determines highlighting language
2. Manual Selection: Use menu to choose specific language
3. Configuration: Access syntax highlighting settings from menu
4. Custom Rules: Load custom configuration files for additional languages

4.4 Code Compilation

1. Prepare Code: Write or open Kotlin code in the editor
2. Compile: Press "Compile & Run" button to start compilation
3. View Output: Check output section for results or errors
4. Debug: Use error information to fix compilation issues

4.5 Advanced Features

1. Find and Replace: Use search functionality for text operations
2. Statistics: View word, character, and line counts from menu
3. Theme Switching: Toggle between dark and light modes
4. File Management: Save, rename, and organize files efficiently

5. Team Members and Contributions

Name	Index Number	Primary Responsibilities
S.D. Senevirathna	23020972	• Leading the overall development process • Set up overall project structure and core architecture • Implemented primary text editor functions (text input area, open/save files, undo/redo, cut/copy/paste) • Built the syntax highlighting engine • Debugged and optimized performance for large files • Built the compilation function for the programming languages
W.K.S.U. Walpola	23021136	• Added word count/code statistics feature • Led the project documentation (README, user guide, feature descriptions) • Planned and scripted the demo video flow and provided voice-over • Coordinated overall team output
S.T. Jayawardhana	23020423	• Implemented line numbering and bracket pairing features • Added file explorer/quick-access panel • Integrated smooth transitions and animations for a better user experience • Assisted in testing and final bug fixes alongside Sachindra

6. Conclusion

In conclusion, this project has provided us with the opportunity to design and develop an Android-based text editor that not only supports essential editing functionalities but also extends to advanced features such as Kotlin syntax highlighting, configurable language support, and integration with ADB for compilation. Through this assignment, we were able to combine theoretical knowledge with practical implementation, enhancing our understanding of mobile application development, compiler integration, and error handling mechanisms.

The outcome demonstrates a functional and user-friendly editor that balances both simplicity and adaptability, catering to developers who wish to quickly test and edit code on Android devices. While certain aspects such as ADB-based compilation involve inherent complexities, our focus remained on delivering a reliable and conceptually sound solution. Overall, this assignment has strengthened our technical skills in Kotlin/Java/Flutter/React, deepened our problem-solving abilities, and provided valuable experience in collaborative project development.